

2. Preparing the code for release using Ant—If Apache Ant isn't already installed, install it using Synaptic. Then run the following command from the directory where the export happened:

```
ant release
```

This will prepare our code for release (previously, Processing was preparing to debug it every time we ran it, signing it with a 'debug' key, which is unlike the 'release' key generated in the previous step). Upon successful build, after generating a lot of data, Ant will finally give the following output:

```
release:
```

```
BUILD SUCCESSFUL
Total time: 15 seconds
```

Our build, i.e., compiled APK, now sits in the `/bin` directory under our export location. Change to the `bin` directory and you will see something like 'myfirstsketch-release-unsigned.apk'. In the next step, we will sign it.

3. Signing the build—Here we sign the build using the key pair we generated. Run the following command in the `/bin` directory within our export directory at the terminal prompt:

```
jarsigner -verbose -keystore ../my-release-key.keystore
myfirstsketch-release-unsigned.apk mykey1
```

The above command will ask for the keystore password we had set while creating the keys and will then show the following output:

```
Enter Passphrase for keystore:
adding: META-INF/MANIFEST.MF
adding: META-INF/MYKEY1.SF
adding: META-INF/MYKEY1.RSA
signing: res/drawable/icon.png
signing: res/layout/main.xml
signing: AndroidManifest.xml
signing: resources.arsc
signing: res/drawable-hdpi/icon.png
signing: res/drawable-ldpi/icon.png
signing: classes.dex
signing: processing/core/FillShaderFragNoTex.gls1
signing: processing/core/FillShaderFragTex.gls1
signing: processing/core/FillShaderVertFull.gls1
signing: processing/core/FillShaderVertLit.gls1
signing: processing/core/FillShaderVertSimple.gls1
signing: processing/core/FillShaderVertTex.gls1
signing: processing/core/LineShaderFrag.gls1
signing: processing/core/LineShaderVert.gls1
```

```
signing: processing/core/PointShaderFrag.gls1
signing: processing/core/PointShaderVert.gls1
```

We can now verify the signature:

```
jarsigner -verify myfirstsketch-release-unsigned.apk
jar verified.
```

4. Wrapping it up—Zipaligning the package is done finally to make sure it runs in an optimised manner on the device. Basically, this step byte-aligns (to 4 bytes, specifically for our purposes) the uncompressed data and readies the APK for deployment. This is the final step before our app/sketch goes out into the world. Run the following command at the terminal (this, too, from the `/bin` directory):

```
zipalign 4 myfirstsketch-release-unsigned.apk myfirstsketch-signed-release.apk
```

The number 4 is specific – do not use any other value. Zipalign resides within the `/tools` directory of the Android SDK—make sure the `/tools` directory is added to the path environment variables to avoid errors. You can alternatively run Zipalign using the following format (from the export `/bin` directory):

```
<path-to-SDK>/zipalign 4 myfirstsketch-release-unsigned.apk
myfirstsketch-signed-release.apk
```

So that's it! Now this APK can be installed on any Android device and even uploaded to Google Play! You need to have a developer account for the latter, but our sketch is now ready to take on the world!

Processing for Android is a cutting-edge piece of software and can be quirky sometimes, but is one super IDE that allows us to make neat visually appealing sketches. After all, the beauty of open source software is to do crazy and awesome stuff! So long for now and Happy Sketching! 

Links

- [1] <http://wiki.processing.org/w/Android>
- [2] <http://www.openprocessing.org>
- [3] <http://developer.android.com/sdk/index.html>
- [4] <http://developer.android.com/guide/publishing/app-signing.html>
- [5] <http://www.oracle.com/technetwork/java/javase/downloads/index.html>
- [6] <http://ant.apache.org/bindownload.cgi>

By: Siddharth Mankad

The author is a new media designer, with an affinity for physical computing, computer software and code, as well as country music.